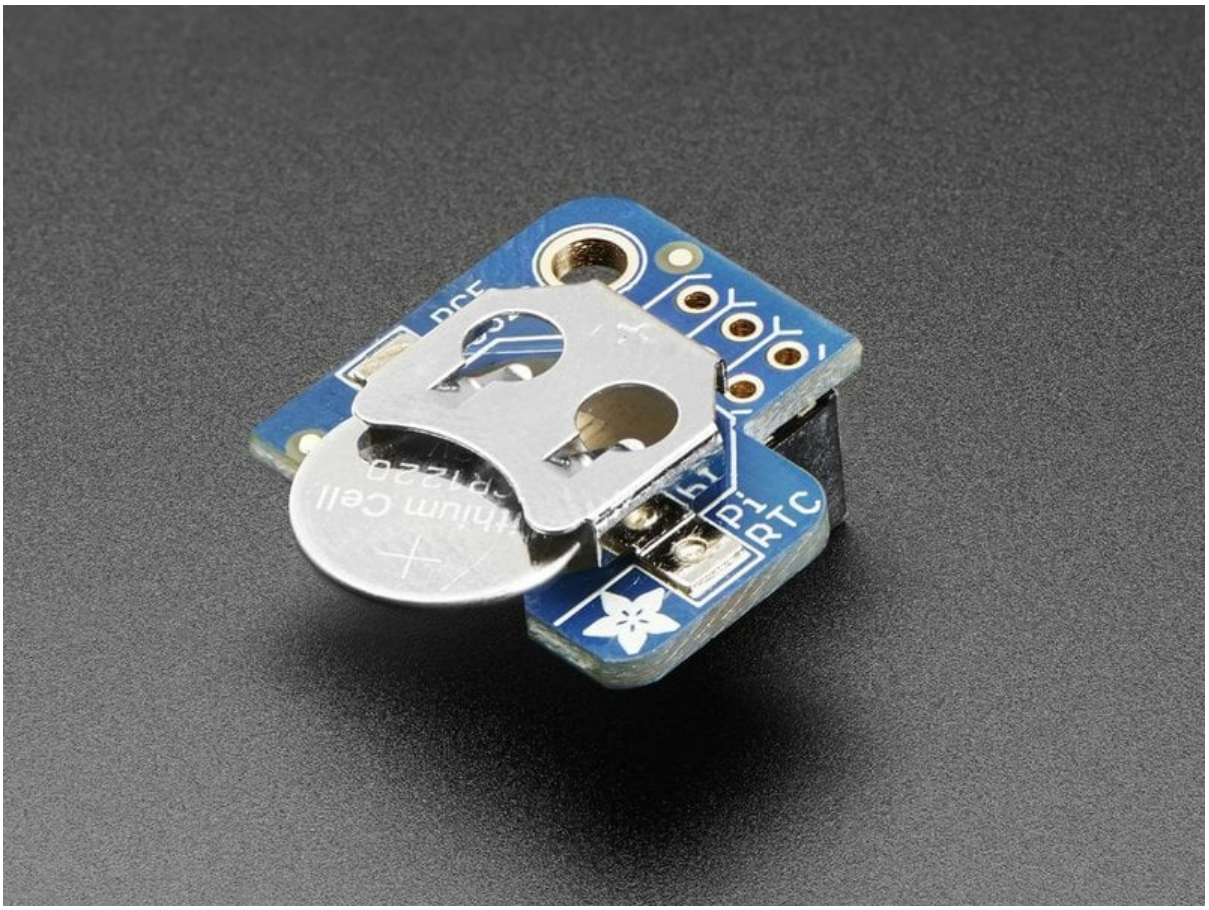
Adding a Real Time Clock to Raspberry Pi

Created by lady ada

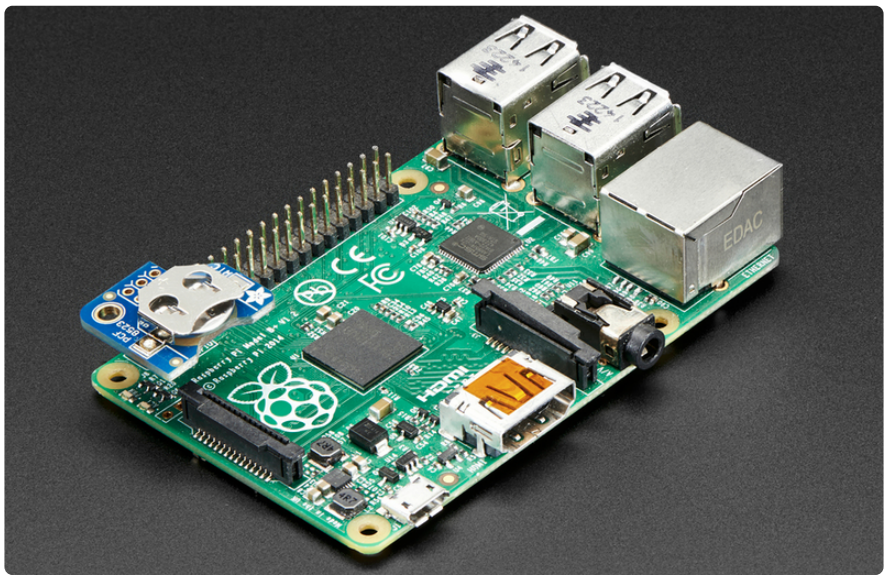


https://learn.adafruit.com/adding-a-real-time-clock-to-raspberry-pi

Last updated on 2026-04-07 08:17:44 PM UTC

Table of Contents

Overview



guide is for Pi models prior to the Pi5, which has a built in RTC. For Pi5 here: https://www.raspberrypi.com/documentation/computers/raspberry-ml#real-time-clock-rtc

The Raspberry Pi is designed to be an ultra-low cost computer, so a lot of things we are used to on a computer have been left out. For example, your laptop and computer have a little coin-battery-powered 'Real Time Clock' (RTC) module, which keeps time even when the power is off, or the battery removed. To keep costs low and the size small, an RTC is not included with the Raspberry Pi. Instead, the Pi is intended to be connected to the Internet via Ethernet or WiFi, updating the time automatically from the global **ntp** (nework time protocol) servers

For stand-alone projects with no network connection, you will not be able to keep the time when the power goes out. So in this project we will show you how to add a low cost battery-backed RTC to your Pi to keep time!

RTC Options

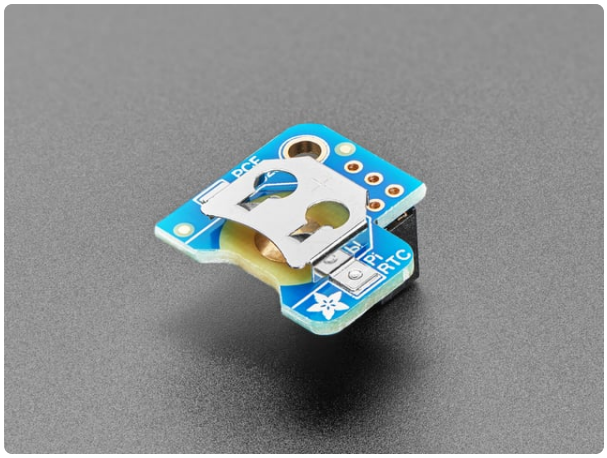
Adafruit currently offers three different RTC options in the shop. As a general summary:

- PCF8523 is inexpensive
- DS3231 is most precise
- DS1307 is historically most common

While the DS1307 is historically the most common, it is not the best RTC chipset, we've found! However, any of these RTC options will function on a Raspberry Pi.

These RTC options also come in various form factors. The oldest style has only a row of headers and requires soldering. Newer style include STEMMA QT connectors to provide a solderless connection option. For the PCF8523 and DS3231, there are also a Pi specific version that will mount directly to the Pi's GPIO header.

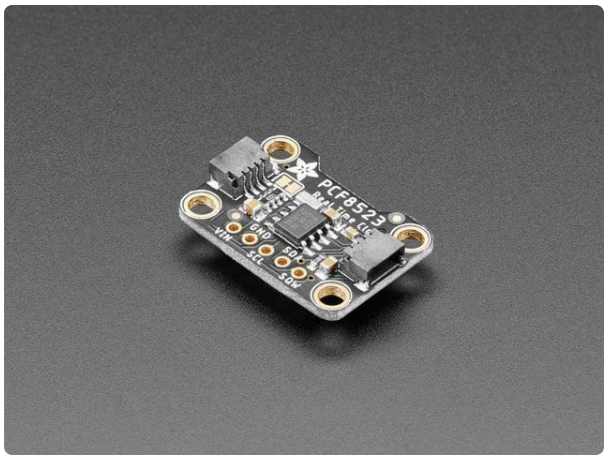
PCF8523



Adafruit PiRTC - PCF8523 Real Time Clock for Raspberry Pi

This is a great battery-backed real time clock (RTC) that allows your Raspberry Pi project to keep track of time if the power is lost. Perfect for data-logging, clock-building,...

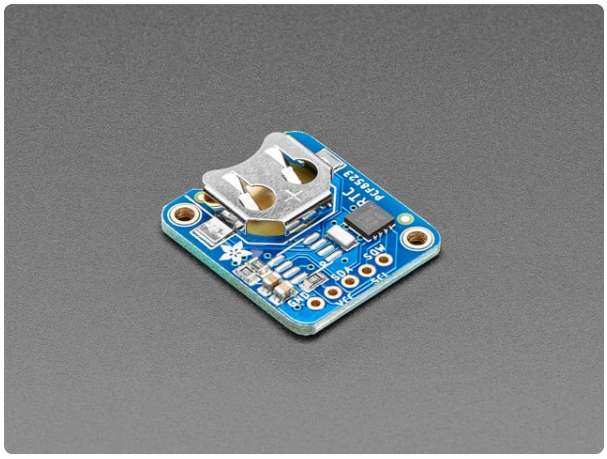
https://www.adafruit.com/product/3386



Adafruit PCF8523 Real Time Clock Breakout Board

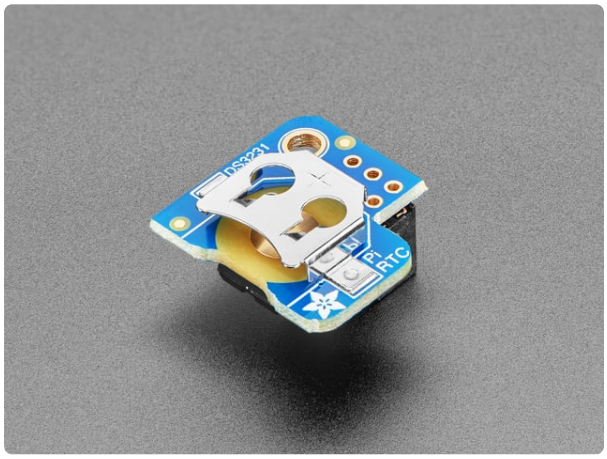
This is a great battery-backed real time clock (RTC) that allows your microcontroller project to keep track of time even if it is reprogrammed, or if the power is lost. Perfect for...

https://www.adafruit.com/product/5189

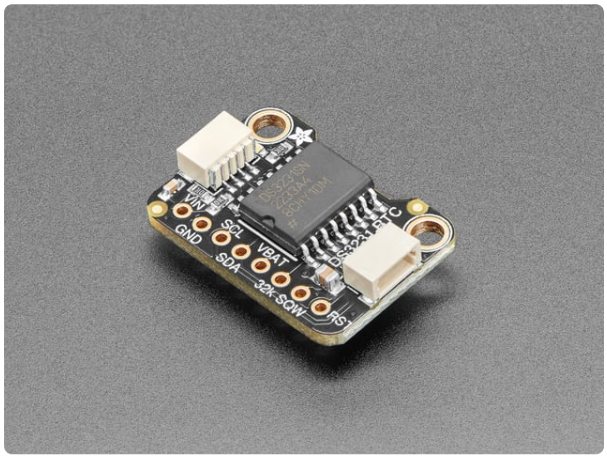


[Adafruit PCF8523 Real Time Clock Assembled Breakout Board]
This is a great battery-backed real time clock (RTC) that allows your microcontroller project to keep track of time even if it is reprogrammed, or if the power is lost. Perfect for...
https://www.adafruit.com/product/3295

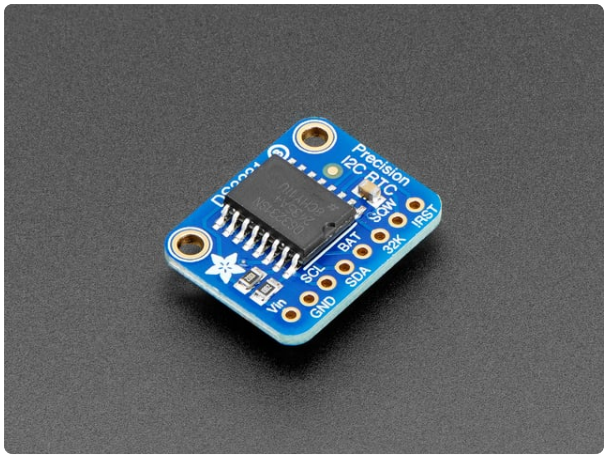
DS3231



[Adafruit PiRTC - Precise DS3231 Real Time Clock for Raspberry Pi]
This is the best battery-backed real time clock (RTC) you can get that allows your Raspberry Pi project to keep track of time if the power is lost. Perfect for data-logging,...
https://www.adafruit.com/product/4282

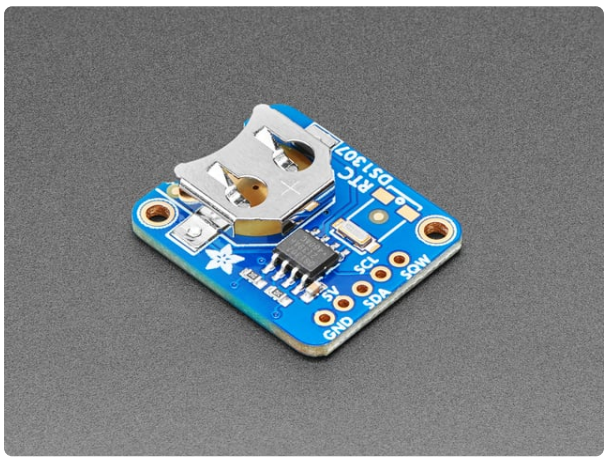


[Adafruit DS3231 Precision RTC - STEMMA QT]
The datasheet for the DS3231 explains that this part is an "Extremely Accurate I^{2}C-Integrated RTC/TCXO/Crystal". And, hey, it does exactly...
https://www.adafruit.com/product/5188



Adafruit DS3231 Precision RTC Breakout
The datasheet for the DS3231 explains that this part is an "Extremely Accurate I²C-Integrated RTC/TCXO/Crystal". And, hey, it does exactly what it says...
https://www.adafruit.com/product/3013

DS1307



Adafruit DS1307 Real Time Clock Assembled Breakout Board
This is a great battery-backed real time clock (RTC) that allows your microcontroller project to keep track of time even if it is reprogrammed, or if the power is lost. Perfect for...
https://www.adafruit.com/product/3296

RTC Battery

Don't forget to also install a CR1220 coin cell. In particular the DS1307 wont work at all without it and none of the RTCs will keep time when the Pi is off and no coin battery is in place.

CR1220 12mm Diameter - 3V Lithium Coin Cell Battery
These are the highest quality & capacity batteries, the same as shipped with the iCufflinks, iNecklace, Datalogging and GPS Shields, GPS HAT, etc. One battery per order...
https://www.adafruit.com/product/380

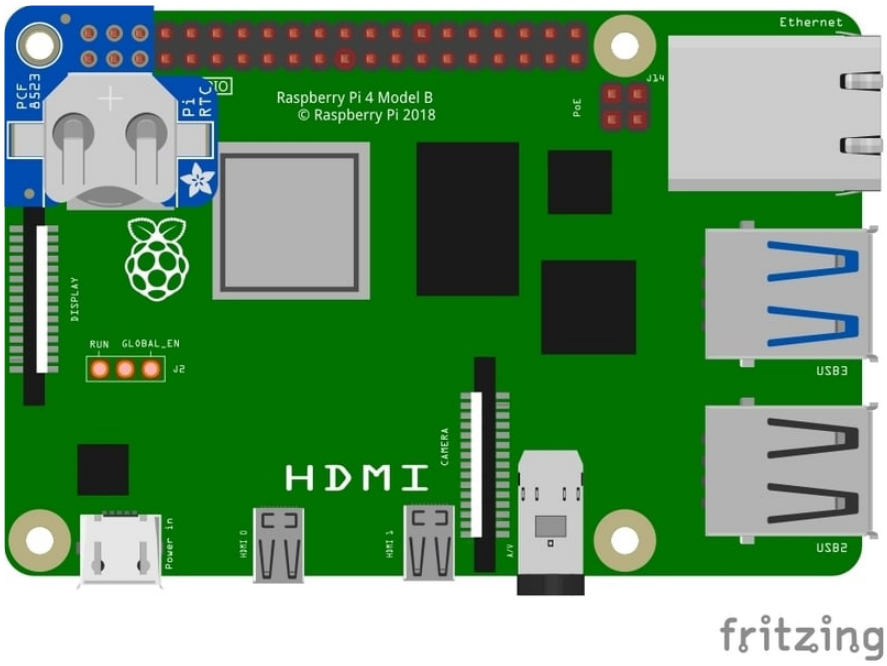
Connecting the RTC

The PCF8523 RTC is used as an example here, but the connections are similar for the other RTC options.

PiRTC Version

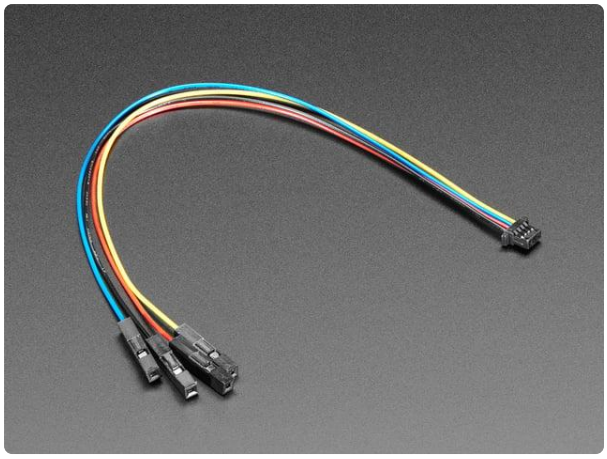
For any of the PiRTC options, just place it on the Pi's GPIO header away from the USB ports.

Be sure to place it on the last 6 pins with the mounting hole of the RTC aligning with the mounting hole of the Pi. Putting it on other pins could harm both boards.



STEMMA QT Breakout Version

For any of the breakout versions with a STEMMA QT connector, the connections can be made without any soldering using a STEMMA QT cable:



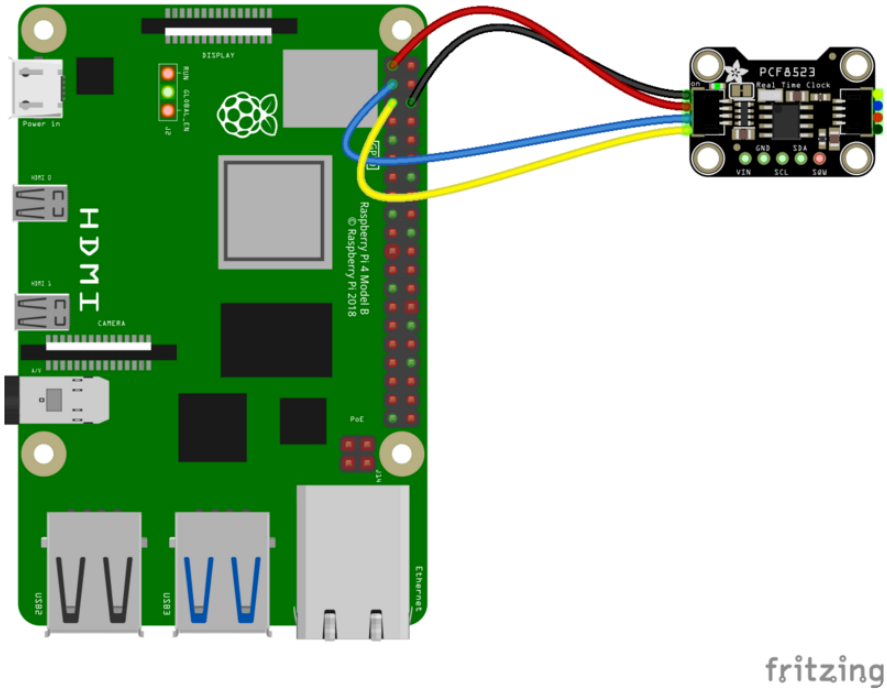
STEMMA QT / Qwiic JST SH 4-pin Cable with Premium Female Sockets

This 4-wire cable is a little over 150mm / 6" long and fitted with JST-SH female 4-pin connectors on one end and premium female headers on the other. Compared with the chunkier...

https://www.adafruit.com/product/4397

The female connectors will connect directly to the Pi's GPIO header pins:

- red wire to 3.3V
- black wire to GND
- blue wire to SDA
- yellow wire to SCL

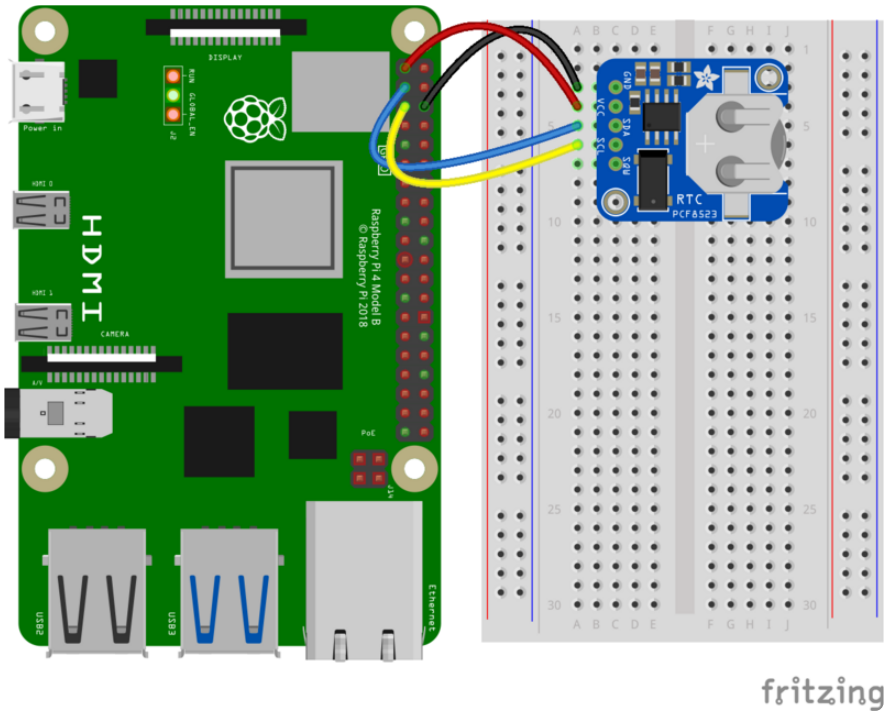


Original Breakout Version

Wiring connections are:

1. Connect **VCC** on the breakout board to the **5.0V** pin of the Pi (if using DS1307)
   Connect **VCC** on the breakout board to the **3.3V** pin of the Pi (if using PCF8523 or DS3231)
2. Connect **GND** on the breakout board to the **GND** pin of the Pi
3. Connect **SDA** on the breakout board to the **SDA** pin of the Pi
4. Connect **SCL** on the breakout board to the **SCL** pin of the Pi

e that soldering is generally required for this setup.



Set Up & Test I2C

You'll also need to set up i2c on your Pi, to do so, run **sudo raspi-config** and under **Interface Options** select I2C and turn it on.

For more details, check out our tutorial on Raspberry Pi i2c setup and testing at http://learn.adafruit.com/adafruits-raspberry-pi-lesson-4-gpio-setup/configuring-i2c (https://adafru.it/aTI)

You may need to reboot once you've done that with **sudo reboot**

Verify Wiring (I2C scan)

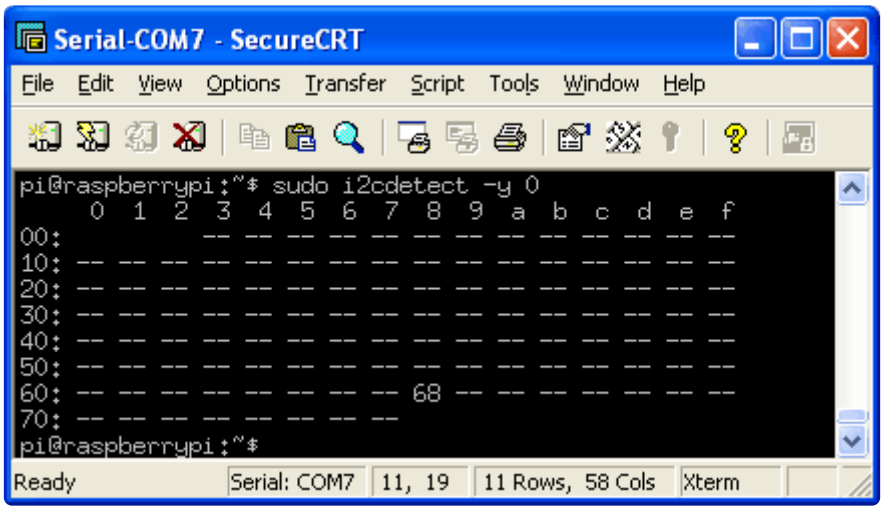
Verify your wiring by running:

```
i2cdetect -y 1
```

If it complains about the command not being found, then it can be installed with:

```
sudo apt install i2c-tools
```

After running `i2cdetect -y 1` at the command line, you should see address 0x68 show up - that's the address of the DS1307, PCF8523 or DS3231!



e you have the kernel driver running, i2cdetect will display UU instead.

If no address shows up in the I2C scan, check your wiring and try the I2C scan again.

Enable RTC and Set Time

Now that we have RTC wired up, the operating system can be configured to use it. This is pretty simple on modern Pi OS versions. The general steps are:

- Enable the appropriate I2C device tree overlay
- Disable time sync service (optional)

following steps were verified using the 2025/12/04 trixie release of the
ɔberry Pi Os.

Enable RTC

Enabling use of the RTC is a simple matter of enabling a device tree overlay. In this case, the **i2c-rtc** overlay is used. The only special thing needed is to specify the actual RTC being used. To enable the overlay, edit the **config.txt** file:

sudo nano /boot/firmware/config.txt

and add one of the following lines to the end, depending on the RTC being used.

For the PCF8523:

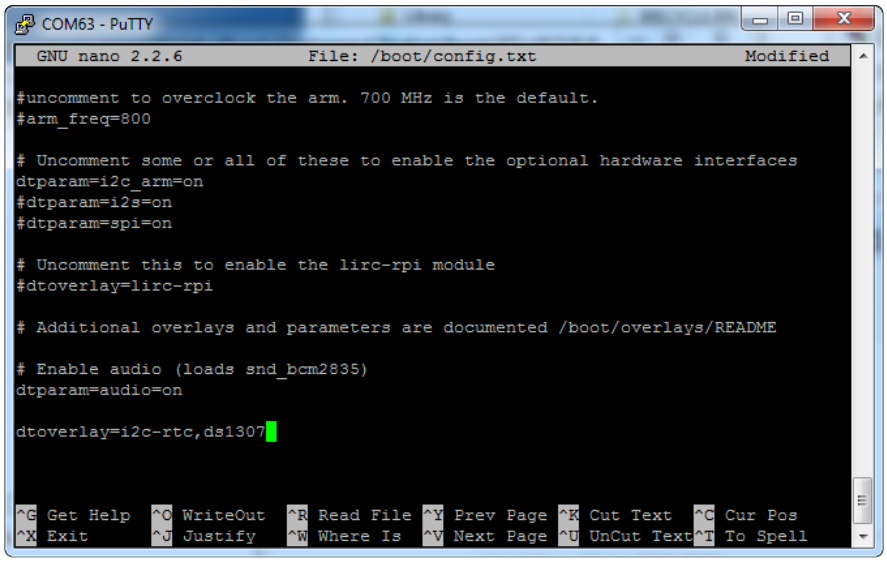
```
dtoverlay=i2c-rtc,pcf8523
```

For the DS3231:

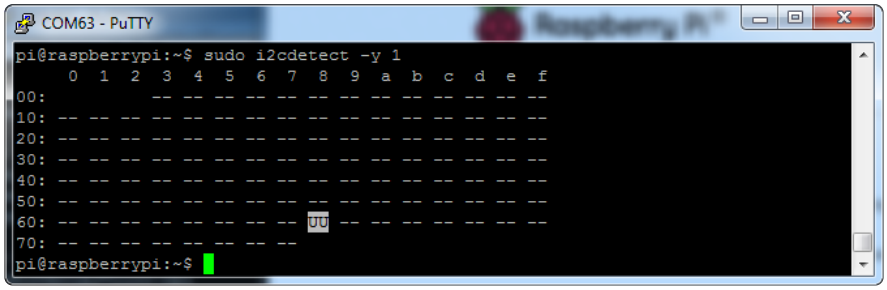
```
dtoverlay=i2c-rtc,ds3231
```

For the DS1307:

```
dtoverlay=i2c-rtc,ds1307
```



Save the changes and run `sudo reboot` to reboot the Pi so the changes take affect. After rebooting, run `i2cdetect -y 1` again to verify that UU now shows up where 0x68 should be:



Setting Time

There are two time sources to consider:

- System Clock - the time according to the operating system
  - use the **date** command to check/set this
- Hardware Clock - the time according to the RTC
  - use the **hwclock** command to check/set this

At boot, the system clock is set to the hardware (RTC) clock.

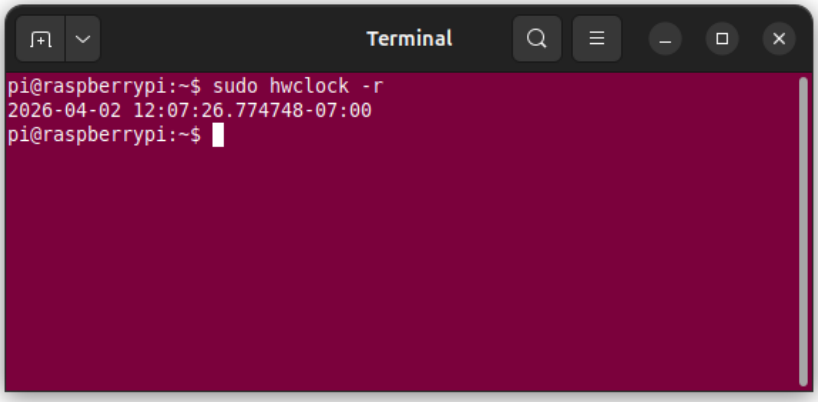
If the Pi has internet access, it will use NTP to sync and set the Pi's time every time it boots up. This will update the RTC as well. See the next section for how to disable this behavior.

There is also a command line tool called **hwclock** which can be used to manually check and set the RTC. It can be installed using:

```
sudo apt install util-linux-extra
```

The current time of the RTC can be read using:

```
sudo hwclock -r
```



To manually set the RTC use:

```
sudo hwclock --set --date='2010-01-02 12:23:42'
```

where the date and time are set using the text string with **YEAR-MONTH-DAY HOUR:MINUTE:SECOND** syntax as shown after the **--date** parameter. Update this for whatever date/time you want.

NOTE: This only sets the RTC time. To set the system time to the updated RTC time, use:

```
sudo hwclock -s
```

This generally only needs to be done once, or anytime the RTC time needs adjusting.

hwclock: ioctl(RTC_RD_TIME) to /dev/rtc0 to read the time failed: Invalid argument

If you are getting an error message like this when trying to read/write to the RTC, make sure you have a good coin cell battery installed.

Disabling Time Sync Service

If you want to disable the time sync service so that the Pi does not attempt to set the time via the internet, run the following:

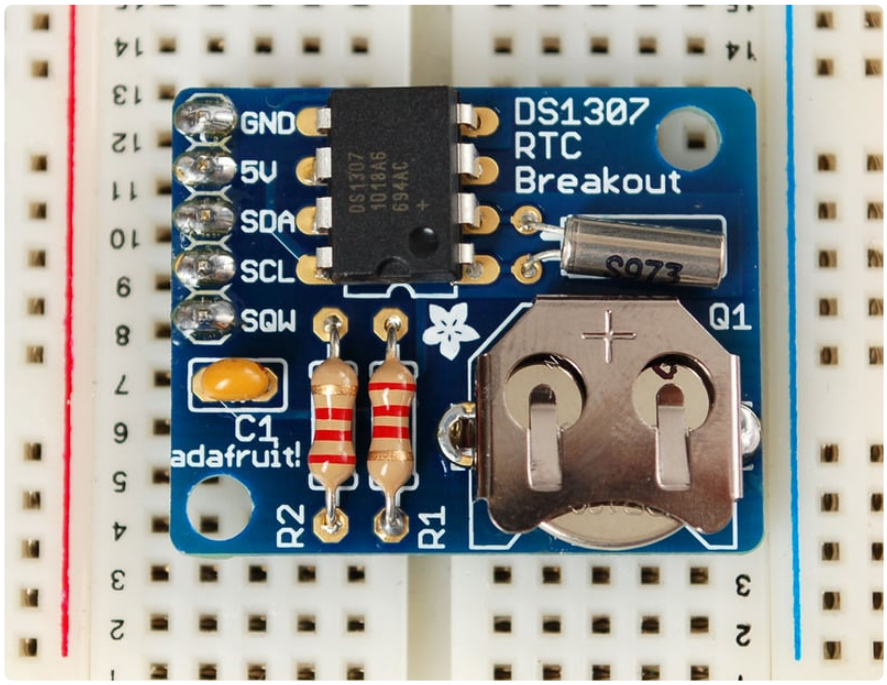
```
sudo systemctl disable systemd-timesyncd
```

Now the Pi should be entirely relying the attached RTC for setting system time.

Older Process (ref only)

These pages are older and are potentially out of date. They are left here for reference in case they are useful when dealing with older OS releases.

Overview



tutorial requires a Raspberry Pi running a kernel with the RTC module and
307 module included. Current Raspbian distros have this, but others may

The Raspberry Pi is designed to be an ultra-low cost computer, so a lot of things we are used to on a computer have been left out. For example, your laptop and computer have a little coin-battery-powered 'Real Time Clock' (RTC) module, which keeps time even when the power is off, or the battery removed. To keep costs low and the size small, an RTC is not included with the Raspberry Pi. Instead, the Pi is intended to be connected to the Internet via Ethernet or WiFi, updating the time automatically from the global **ntp** (nework time protocol) servers

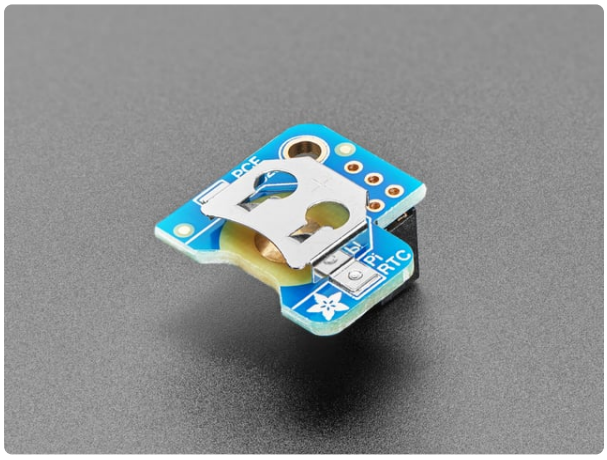
For stand-alone projects with no network connection, you will not be able to keep the time when the power goes out. So in this project we will show you how to add a low cost battery-backed RTC to your Pi to keep time!

Wiring the RTC

To keep costs low, the Raspberry Pi does not include a Real Time Clock module. Instead, users are expected to have it always connected to WiFi or Ethernet and keep time by checking the network. Since we want to include an external module, we'll have to wire one up.

We have three different RTC we suggest, PCF8523 is inexpensive, DS1307 is most common, and DS3231 is most precise. Any of them will do!

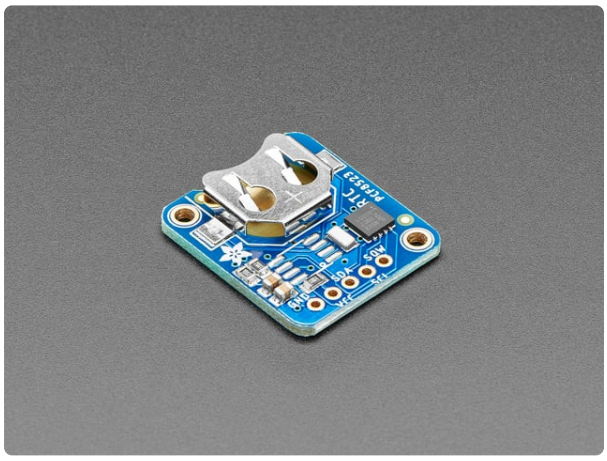
While the DS1307 is historically the most common, its not the best RTC chipset, we've found!



Adafruit PiRTC - PCF8523 Real Time Clock for Raspberry Pi

This is a great battery-backed real time clock (RTC) that allows your Raspberry Pi project to keep track of time if the power is lost. Perfect for data-logging, clock-building,...

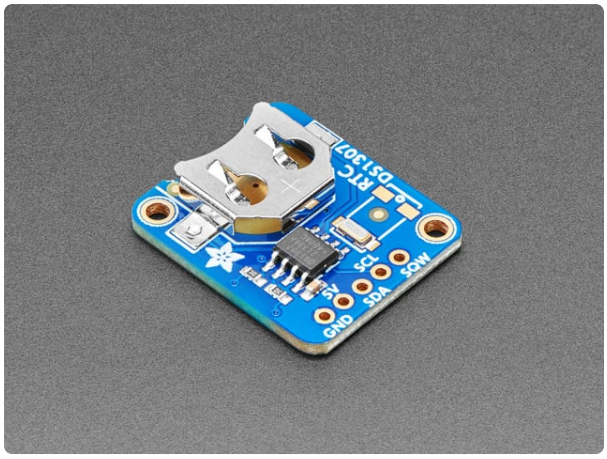
https://www.adafruit.com/product/3386



Adafruit PCF8523 Real Time Clock Assembled Breakout Board

This is a great battery-backed real time clock (RTC) that allows your microcontroller project to keep track of time even if it is reprogrammed, or if the power is lost. Perfect for...

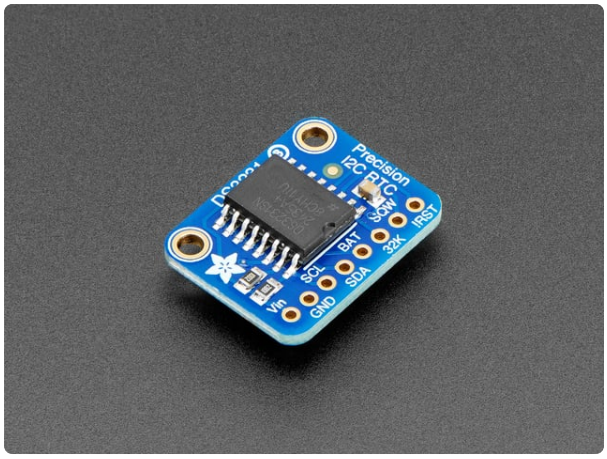
https://www.adafruit.com/product/3295



[Adafruit DS1307 Real Time Clock Assembled Breakout Board](https://www.adafruit.com/product/3296)

This is a great battery-backed real time clock (RTC) that allows your microcontroller project to keep track of time even if it is reprogrammed, or if the power is lost. Perfect for...

https://www.adafruit.com/product/3296



[Adafruit DS3231 Precision RTC Breakout](https://www.adafruit.com/product/3013)

The datasheet for the DS3231 explains that this part is an "Extremely Accurate I²C-Integrated RTC/TCXO/Crystal". And, hey, it does exactly what it says...

https://www.adafruit.com/product/3013

Don't forget to also install a CR1220 coin cell. In particular the DS1307 wont work at all without it and none of the RTCs will keep time when the Pi is off and no coin battery is in place.

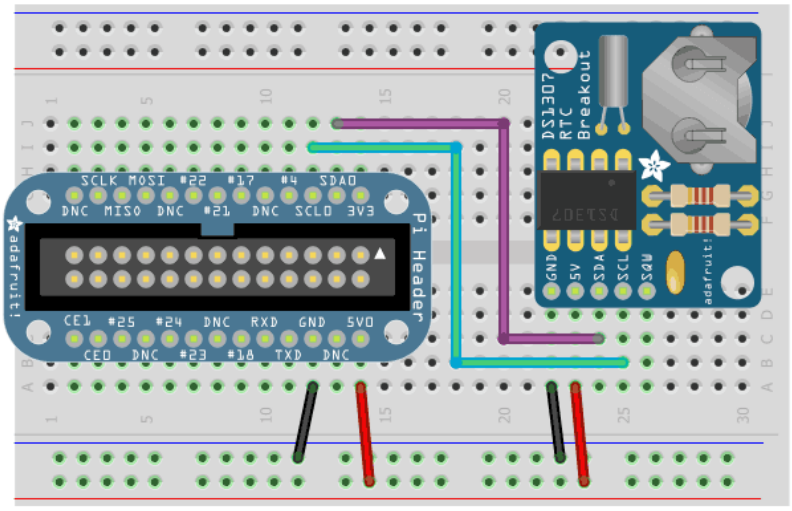
[CR1220 12mm Diameter - 3V Lithium Coin Cell Battery](https://www.adafruit.com/product/380)

These are the highest quality & capacity batteries, the same as shipped with the iCufflinks, iNecklace, Datalogging and GPS Shields, GPS HAT, etc. One battery per order...

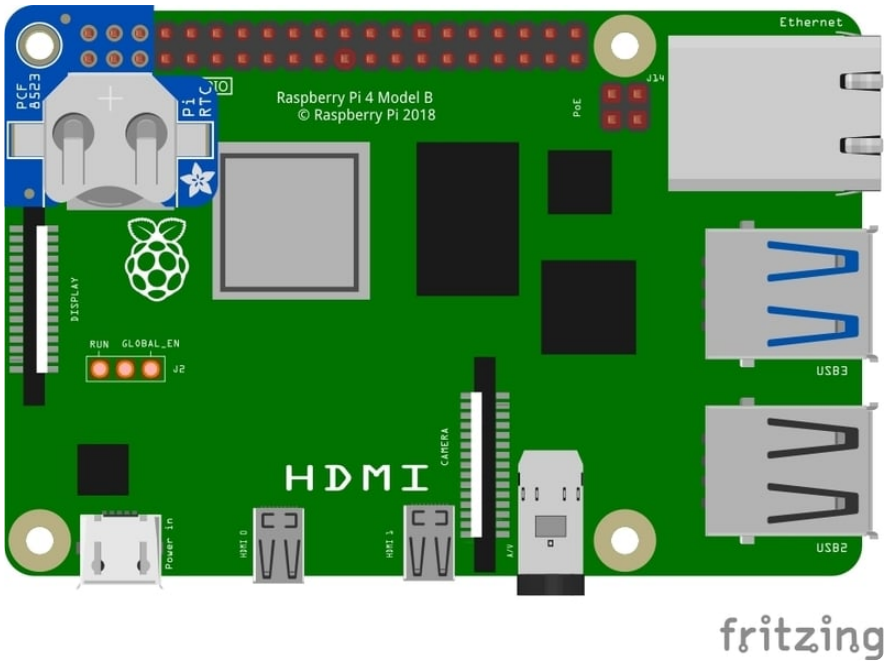
https://www.adafruit.com/product/380

Wiring is simple:

1. Connect **VCC** on the breakout board to the **5.0V** pin of the Pi (if using DS1307)
   Connect **VCC** on the breakout board to the **3.3V** pin of the Pi (if using PCF8523 or DS3231)
2. Connect **GND** on the breakout board to the **GND** pin of the Pi
3. Connect **SDA** on the breakout board to the **SDA** pin of the Pi
4. Connect **SCL** on the breakout board to the **SCL** pin of the Pi



For the PiRTC, just place it on the header away from the USB ports.



Set Up & Test I2C

Set up I2C on your Pi

You'll also need to set up I2C on your Pi, to do so, run **sudo raspi-config** and under **Interface Options**, select I2C and turn it on.

For more details, check out our tutorial on Raspberry Pi i2C setup and testing at http://learn.adafruit.com/adafruits-raspberry-pi-lesson-4-gpio-setup/configuring-i2c (https://adafru.it/aTI)

You may need to reboot once you've done that with **sudo reboot**

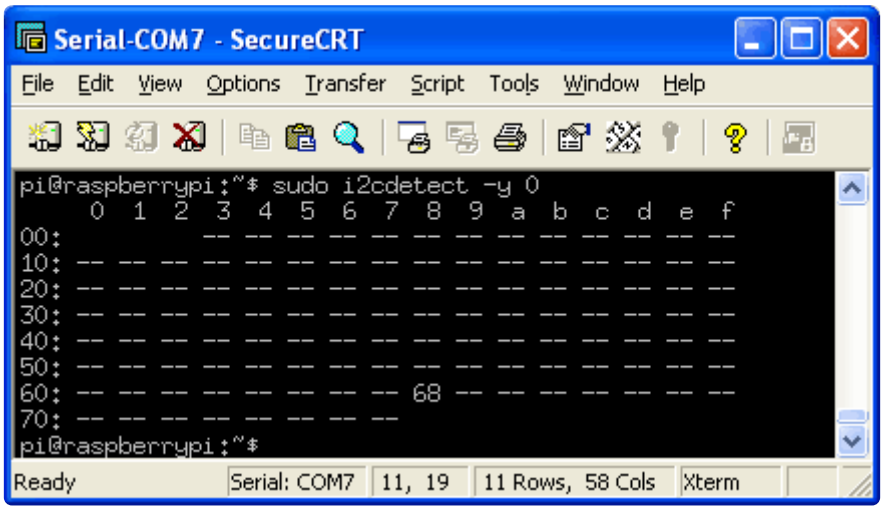
Verify Wiring (I2C scan)

Verify your wiring by running

```
sudo apt-get install python3-smbus i2c-tools
```

This installs the helper software and then `sudo i2cdetect -y 1` at the command line, you should see ID #68 show up - that's the address of the DS1307, PCF8523 or DS3231!

If you have a much older Pi 1, you will have to run `sudo i2cdetect -y 0` as the I2C bus address changed from 0 to 1.



e you have the Kernel driver running, i2cdetect will skip over 0x68 and lay UU instead, this means its working!

Set RTC Time

Now that we have the module wired up and verified that you can see the module with i2cdetect, we can set up the module.

't forget to set up I2C in the previous step!

Raspberry Pi OS's with systemd

This should be the case for any current release. For much older releases without systemd, skip to the next section.

[Thanks to ad8g for the hints!](https://adafru.it/1afz) (https://adafru.it/1afz)

You can add support for the RTC by adding a device tree overlay. Run

sudo nano /boot/config.txt

to edit the pi configuration and add whichever matches your RTC chip:

```
dtoverlay=i2c-rtc,ds1307
```

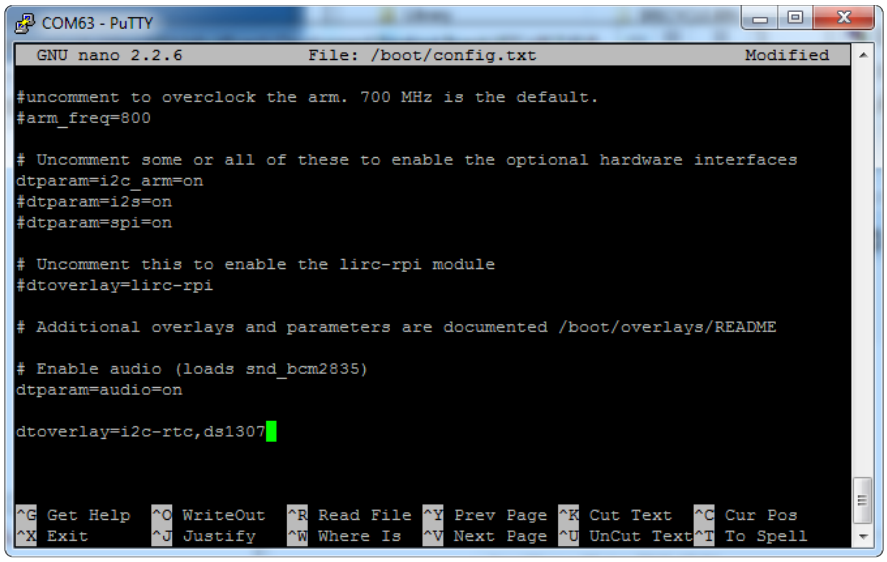
or

```
dtoverlay=i2c-rtc,pcf8523
```

or

```
dtoverlay=i2c-rtc,ds3231
```

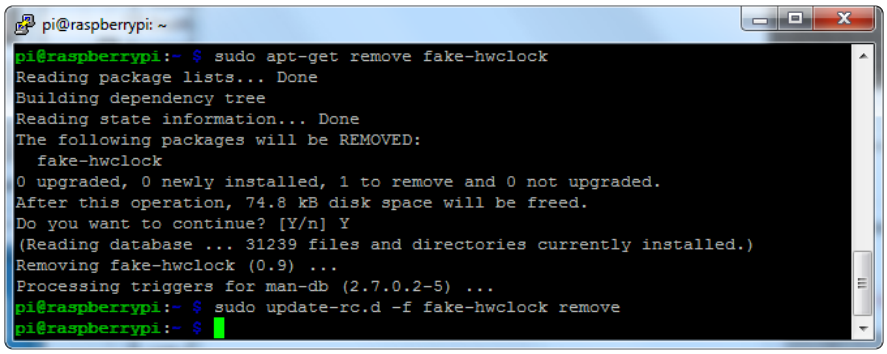
to the end of the file



Save it and run `sudo reboot` to start again. Log in and run `sudo i2cdetect -y 1` to see the UU show up where 0x68 should be

Disable the "fake hwclock" which interferes with the 'real' hwclock

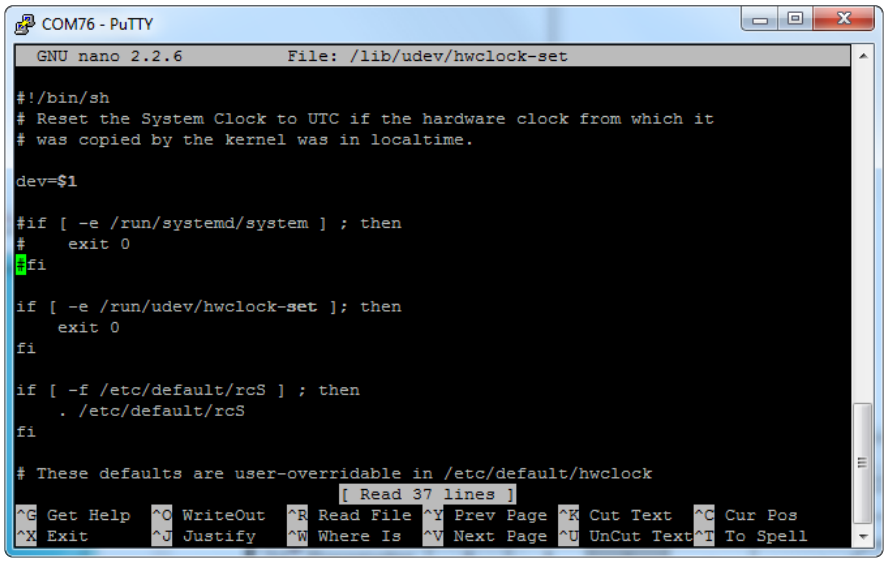
```
sudo apt-get -y remove fake-hwclock
sudo update-rc.d -f fake-hwclock remove
sudo systemctl disable fake-hwclock
```



Now with the fake-hw clock off, you can start the original 'hardware clock' script.

Run **sudo nano /lib/udev/hwclock-set** and comment out these three lines:

#if [-e /run/systemd/system] ; then
 # exit 0
#fi



Also comment out the two lines

`/sbin/hwclock --rtc=$dev --systz --badyear`

and

`/sbin/hwclock --rtc=$dev --systz`

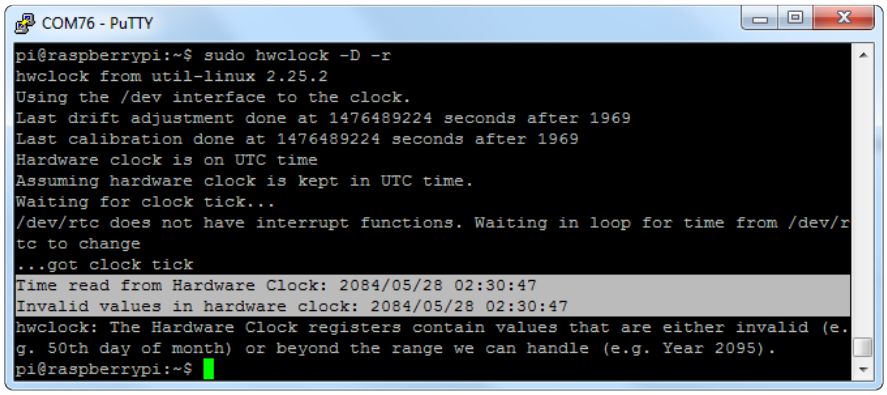
```
if [ yes = "$BADYEAR" ] ; then
#     /sbin/hwclock --rtc=$dev --systz --badyear
    /sbin/hwclock --rtc=$dev --hctosys --badyear
else
#     /sbin/hwclock --rtc=$dev --systz
    /sbin/hwclock --rtc=$dev --hctosys
fi

# Note 'touch' may not be available in initramfs
> /run/udev/hwclock-set
```

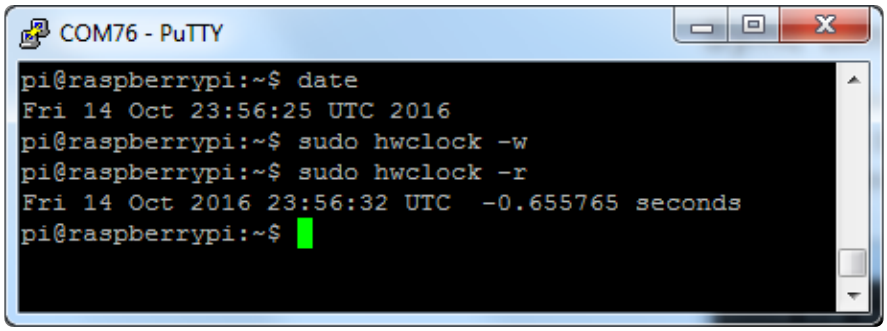
Sync time from Pi to RTC

When you first plug in the RTC module, it's going to have the wrong time because it has to be set once. You can always read the time directly from the RTC with `sudo hwclock -r`

(ignore use of deprecated -D parameter in the screenshot)



You can see, the date at first is invalid! You can set the correct time easily. First run `date` to verify the time is correct. Plug in Ethernet or WiFi to let the Pi sync the right time from the Internet. Once that's done, run `sudo hwclock -w` to write the time, and another `sudo hwclock -r` to read the time



Once the time is set, make sure the coin cell battery is inserted so that the time is saved. You only have to set the time once.

That's it! Next time you boot the time will automatically be synced from the RTC module

To only ready time from the RTC and not the internet, you can disable the timesync service with:

```
sudo systemctl disable systemd-timesyncd.service
```

hwclock: ioctl(RTC_RD_TIME) to /dev/rtc0 to read the time failed: Invalid argument

If you are getting an error message like this when trying to read/write to the RTC, make sure you have a good coin cell battery installed.

Raspbian Wheezy or other pre-systemd Linux

First, load up the RTC module by running

```
sudo modprobe i2c-bcm2708
sudo modprobe i2c-dev
sudo modprobe rtc-ds1307
```

Then, as root (type in **sudo bash**) run

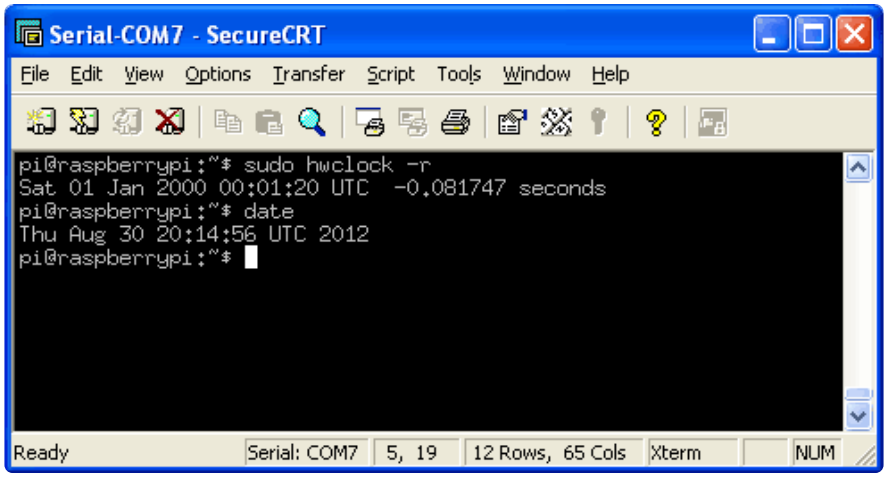
```
echo ds1307 0x68 > /sys/class/i2c-adapter/i2c-1/new_device
```

If you happen to have an old Rev 1 Pi, type in

```
echo ds1307 0x68 > /sys/class/i2c-adapter/i2c-0/new_device
```

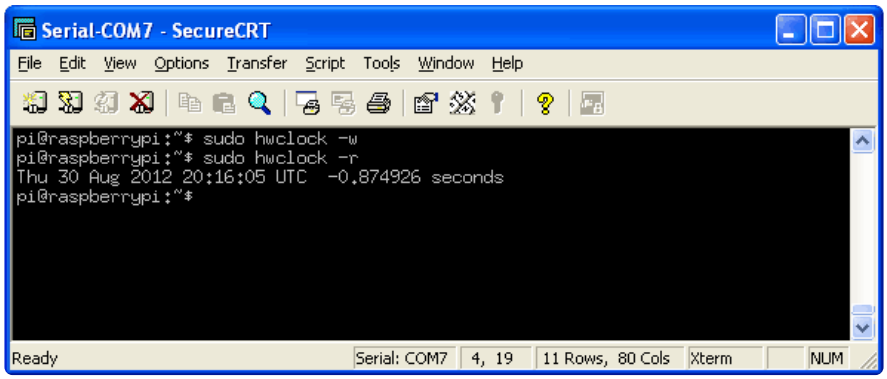
You can then type in **exit** to drop out of the root shell.

Then check the time with **sudo hwclock -r** which will read the time from the DS1307 module. If this is the first time the module has been used, it will report back Jan 1 2000, and you'll need to set the time

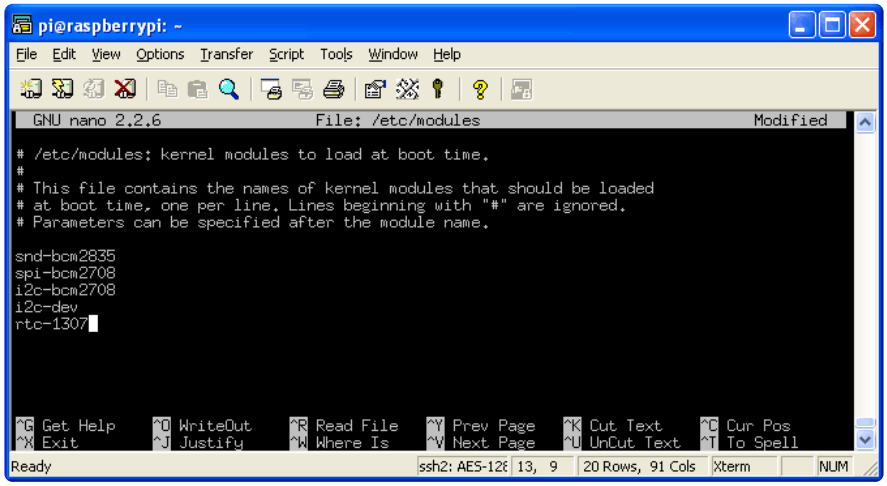


First you'll need to get the right time set on the Pi, the easiest way is to connect it up to Ethernet or WiFi - it will automatically set the time from the network. Once the time is correct (check with the **date** command), run **sudo hwclock -w** to write the system time to the RTC

You can then verify it with `sudo hwclock -r`.



Next, you'll want to add the RTC kernel module to the **/etc/modules** list, so it's loaded when the machine boots. Run `sudo nano /etc/modules` and add **rtc-ds1307** at the end of the file (the image below says rtc-1307 but it's a typo).



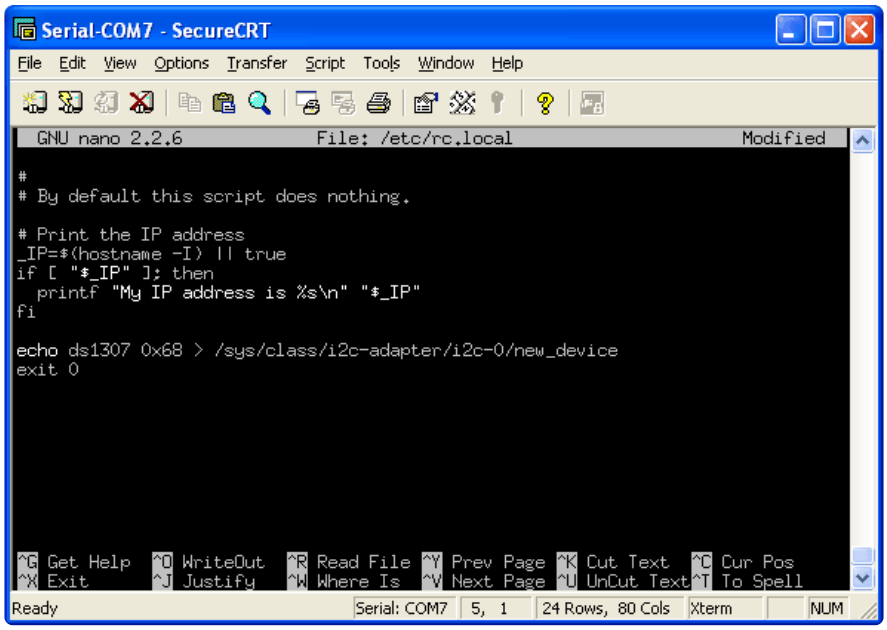
Older pre-Jessie raspbian is a little different. First up, you'll want to create the DS1307 device creation at boot, edit /etc/rc.local by running

```
sudo nano /etc/rc.local
```

and add:

```
echo ds1307 0x68 > /sys/class/i2c-adapter/i2c-0/new_device
```
(for v1 raspberry pi)
```
echo ds1307 0x68 > /sys/class/i2c-adapter/i2c-1/new_device
```
(for v2 raspberry pi)
`sudo hwclock -s` (both versions)

before `exit 0` (we forgot the hwclock -s part in the screenshot below).



That's it! Next time you boot the time will automatically be synced from the RTC module